

2026_Neural Network_ Lecture 1 notes

Neural Networks

BERN02 — Scientific data handling and data fitting, Lund University

Two lectures and two labs.

	Lecture	Lab
1	From the neuron to back-propagation: what a network is and how it learns	<code>NeuralNetworks_1.ipynb</code> — a two-layer network in numpy, a convolutional network in Keras, faces, MNIST
2	Making it work: optimizers, regularization, convolutions, attention	<code>NeuralNetworks_2.ipynb</code> — optimizers, regularization, a 1-D CNN and a tiny transformer on synthetic spectra

Reading

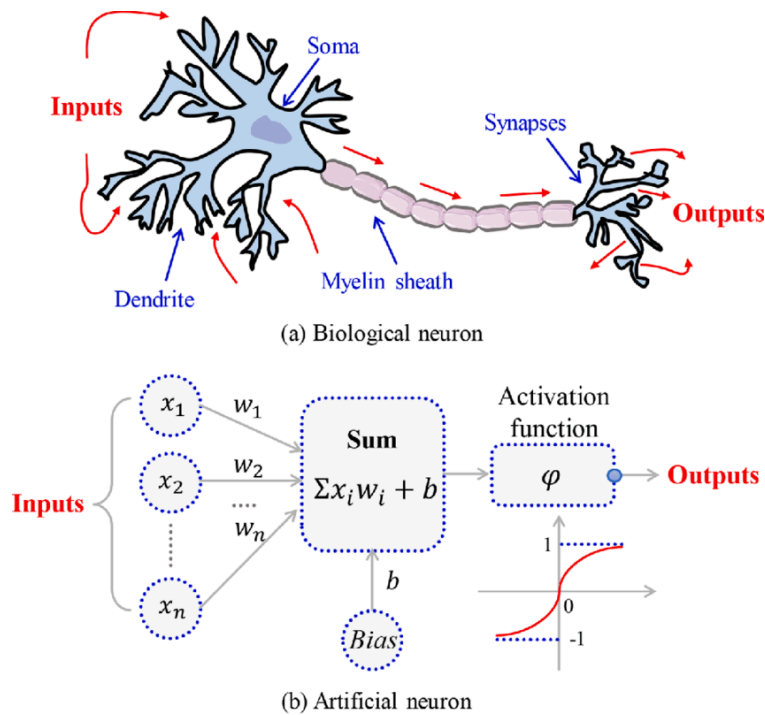
- Bishop & Bishop, *Deep Learning: Foundations and Concepts* (2024), chapters 6–9 and 12. Free online: <https://www.bishopbook.com>
- Prince, *Understanding Deep Learning* (2023), chapters 6–7, 10, 12. Free online: <https://udlbook.github.io/udlbook/>
- Goodfellow, Bengio & Courville, *Deep Learning* (2016), chapters 6–9. Free online: <https://www.deeplearningbook.org>
- James, Witten, Hastie, Tibshirani & Taylor, *An Introduction to Statistical Learning with Python*, chapter 10. <https://www.statlearning.com>
- 3Blue1Brown, *Neural networks* video series (the back-propagation and attention episodes). <https://www.3blue1brown.com/topics/neural-networks>

Lecture 1 — From the neuron to back-propagation

Introduction to Neural Networks

- Biological inspiration: neurons, synapses, layers of cells
- Artificial components: nodes, weighted edges, layers (input, hidden, output)
- What is actually there: a large, flexible fitting function with 10^4 to 10^{11} parameters, and a way to fit it
- Everything you know about curve fitting from this course applies, with one exception that we come to in Lecture 2: we cannot afford the Hessian (second derivative, that is used in gradient decent).

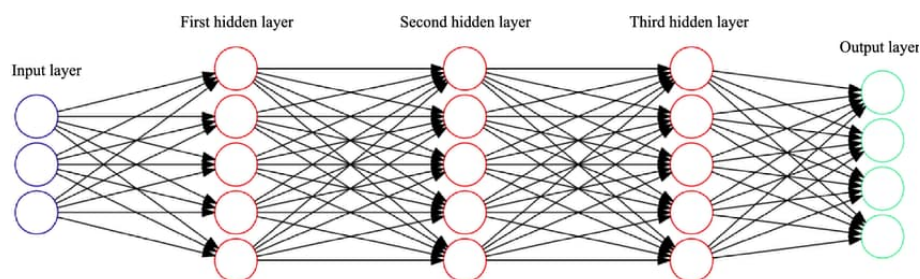
1M params = 4MB for gradient (P) and 4 TB for hessian (P^2)



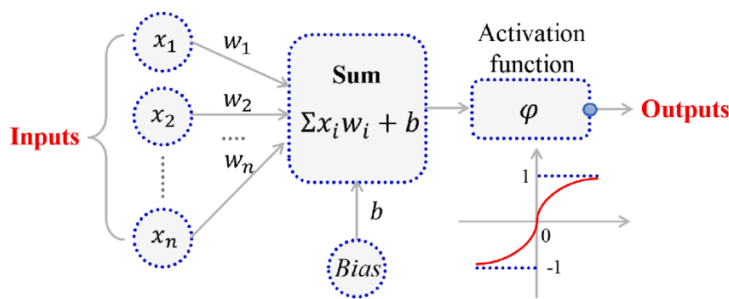
<https://www.researchgate.net/profile/Xianlin-Wang/publication/351372032/figure/fig4/AS:1020744041525248@1620375752492/Comparison-between-biological-neuron-and-artificial-neuron-40.png>;

Basic Architecture of Neural Networks

- Input layer: receives the data; normalize it first
- Hidden layers: one in a "shallow" network, many in a "deep" one
- Output layer: one number for regression, one probability per class for classification
- Dense (fully connected) layers connect every unit to every unit of the previous layer
- One unit: $z_j^{(l)} = \sum_i w_{ji}^{(l)} a_i^{(l-1)} + b_j^{(l)}, \quad a_j^{(l)} = h(z_j^{(l)})$



https://www.researchgate.net/figure/Fully-connected-deep-neural-network_fig1_332197484



<https://www.researchgate.net/profile/Xianlin-Wang/publication/351372032/figure/fig4/AS:1020744041525248@1620375752492/Comparison-between-biological-neuron-and-artificial-neuron-40.png>

One unit is a weighted sum followed by a nonlinearity:

$$a_j = \sum_i w_{ji} x_i + b_j \text{ and } z_j = h(a_j)$$

Without h , a stack of layers would collapse into one linear map. The nonlinearity is what makes depth useful.

Practical Applications of Neural Networks

- Shape fitting: regression, curve fitting, function approximation
 - Classification: binary, multi-class, hierarchical
 - Denoising, super-resolution, inpainting of images and spectra
 - Everyday examples: image recognition, speech recognition, language models
 - In chemistry and physics: spectrum-to-structure, force fields (SchNet, MACE), protein structure (AlphaFold), detector triggers
-

Activation Functions

- Properties: nonlinear, differentiable (almost everywhere), bounded or unbounded range
- Sigmoid and tanh saturate: for large $|a|$ the derivative is nearly zero, and the gradient vanishes
- ReLU $h(a) = \max(0, a)$ does not saturate, but a unit with $a < 0$ for every input is "dead" and never learns
- Leaky ReLU $h(a) = \max(\alpha a, a)$ keeps a small slope to avoid dead units
- Default choice today: ReLU or a smooth cousin (GELU, SiLU) in hidden layers; sigmoid or softmax only in the output layer

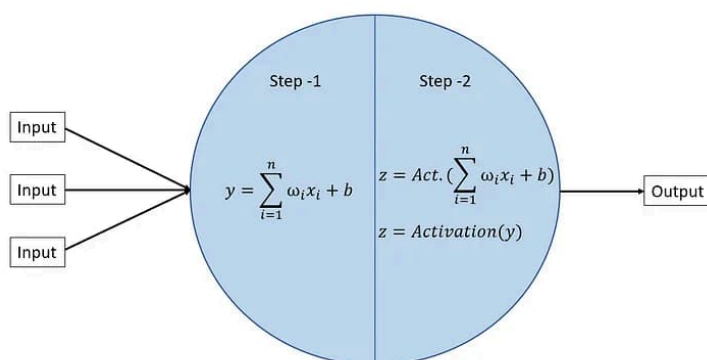
Name	Plot	Equation	Derivative
Identity		$f(x) = x$	$f'(x) = 1$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
Tanh		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parametric Rectified Linear Unit (PReLU) ^[2]		$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Exponential Linear Unit (ELU) ^[3]		$f(x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
SoftPlus		$f(x) = \log_e(1 + e^x)$	$f'(x) = \frac{1}{1 + e^{-x}}$

https://miro.medium.com/max/1400/1*p_hyqAyl8pbt2kEl6siOQ.png

Lab 1, Part A: sigmoid hidden layer. Lab 2, Task 2.2: tanh versus ReLU on a spectrum.

Forward Propagation

- Input $\mathbf{x}$ enters the first layer
- Each layer: weighted sum, then activation, $\mathbf{a}^{(l)} = h(W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$
- The last layer produces the prediction $\hat{y}$
- With a batch of inputs, each step is one matrix multiplication — that is why GPUs help



<https://medium.com/analytics-vidhya/what-do-you-mean-by-forward-propagation-in-ann-9a89c80dac1b&h>

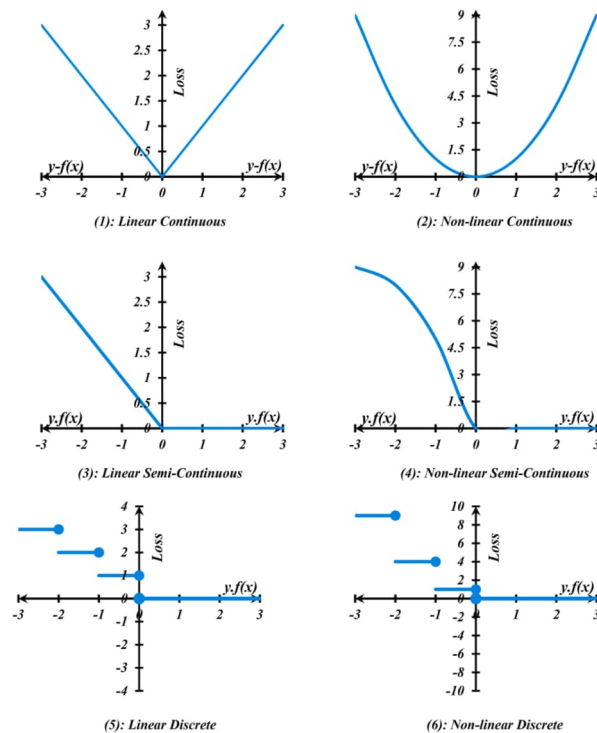
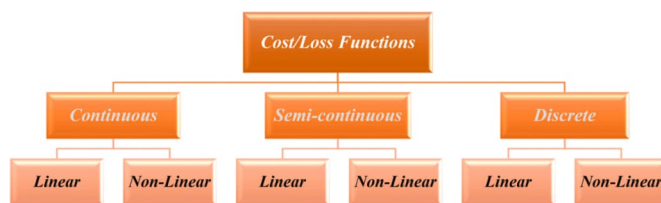
Lab 1, Part A: `Z1 = W1 @ X + b1; A1 = sigmoid(Z1); ...`

Loss Functions

The loss measures how far the prediction is from the target. Training minimizes it.

1. Mean squared error (regression): $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$ — the χ^2 of curve fitting
2. Mean absolute error: $MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$ — robust against outliers
3. Binary cross-entropy (two classes): $BCE = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$
4. Categorical cross-entropy (many classes): $CCE = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^m y_{ij} \log \hat{y}_{ij}$

Cross-entropy is the negative log-likelihood of the labels under the predicted probabilities. A network that knows nothing about m balanced classes has a loss of $\ln m$; check this at the start of training.

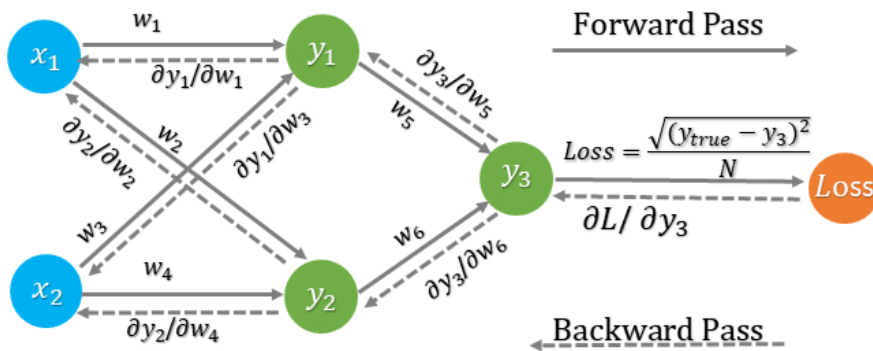


<https://www.sciencedirect.com/science/article/pii/S0952197623015993#fig1&it>

Backpropagation and Training

- The gradient of the loss with respect to every weight is computed by the chain rule, layer by layer from the output backwards
- For the output layer, $\delta_j = \partial L / \partial a_j$ is simple; for the layer before, $\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$
- Then $\partial L / \partial w_{ji} = \delta_j z_i$: the gradient of a weight is the error at its output times the activity at its input

- Gradient descent: $w \leftarrow w - \eta \partial L / \partial w$ with the learning rate η
- Cost: one backward pass costs about two forward passes, whatever the number of parameters



https://www.researchgate.net/figure/An-overview-of-backpropagation-a-Role-of-backpropagation-in-a-neural-network-b_fig2_356390636&it

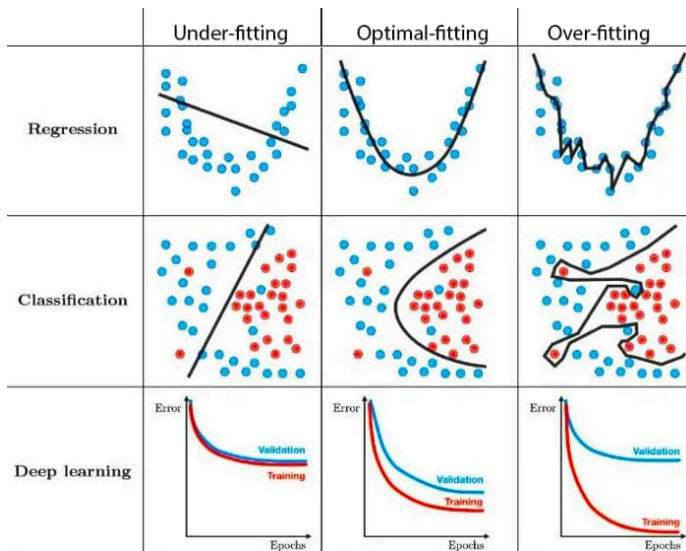
Lab 1, Part A: $dZ2 = A2 - Y$; $dW2 = dZ2 @ A1.T / m$; $dZ1 = (W2.T @ dZ2) * A1 * (1 - A1$
Simple to implement even in **EXCEL** ->Show

Practical considerations and terms

1. Training, validation and test data: split at random; tune on validation, report on test, never touch the test set before the end
2. Epoch: one pass through the training data
3. Batch: the subset of the training data used for one gradient step
4. Learning rate: the step length; the first hyperparameter to tune
5. Learning-rate decay: smaller steps later in training (Lab 1 multiplies the rate by 0.999 per iteration)
6. Dropout rate: the fraction of units switched off at random during training, a cure for overfitting (Lecture 2)
7. Data augmentation: making new training examples from old ones by transformations that do not change the label

Overfitting and Underfitting

- Bias–variance trade-off: too simple a model misses the structure, too complex a model fits the noise
- Capacity versus data set size: a network with 10^5 parameters memorizes 60 examples with ease
- Diagnosis: plot training and validation loss against the epoch. Overfitting is a validation loss that rises while the training loss keeps falling
- Early stopping: stop when the validation loss stops improving and keep the best weights



<https://towardsdatascience.com/techniques-for-handling-underfitting-and-overfitting-in-machine-learning-348daa2380b9&it>

Lab 1, Part B: the CNN with and without dropout, and `EarlyStopping(restore\_best\_weights=True)`.

Practical Implementation

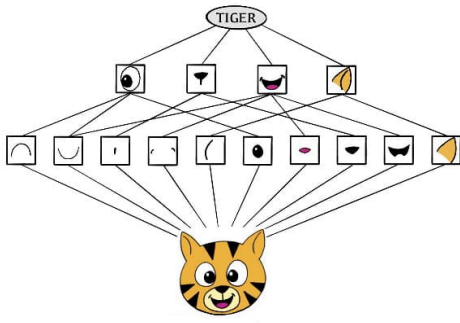
- Frameworks: PyTorch, TensorFlow, Keras (a high-level API that runs on both), JAX
- Components: model definition, data loading, training loop, evaluation
- Automatic differentiation: you write the forward pass, the framework writes the backward pass
- GPU acceleration: CUDA, cuDNN; Colab gives you a free GPU
- Debugging: gradient checking, loss-curve plots, learning-rate scans

Not professional but instructive: numpy (Lab 1), Excel.

Convolutional networks, a first look

Lab 1 uses a convolutional network (CNN) for the bird/dog images. What you need for the lab:

- A convolution layer slides a small kernel (3×3 pixels) over the image and computes the same weighted sum at every position. The kernel weights are learned
- A pooling layer halves the resolution by taking the maximum in each 2×2 block
- Stacks of convolution + pooling build features from edges to shapes to objects
- A dense layer with 3072 inputs and 128 units has 393 000 parameters. A convolution layer with 32 kernels of 3×3×3 has 896. Lecture 2 explains why that matters



James et al., ISL, Figure 10.6

Lecture 1 — Summary

- A network is a chain of weighted sums and nonlinearities; the nonlinearity makes depth useful
- Training is curve fitting with a very large number of parameters: choose a loss, compute its gradient by back-propagation, walk downhill
- Sigmoid and tanh saturate, ReLU does not; the output layer decides the loss (MSE, cross-entropy)
- Split the data, watch the validation loss, stop early

Learning questions

1. Name the essential components of a network and the function of each layer. Explain the terms epoch, batch and learning rate
2. Why does a network need an activation function? Compare sigmoid, tanh and (leaky) ReLU and state when you would use which
3. Name the main loss functions and say what each measures. What is the initial value of the cross-entropy for m balanced classes, and why?
4. Explain back-propagation in words and with the formula $\partial L / \partial w_{ji} = \delta_j z_i$. Where does the vanishing gradient come from?
5. How do you recognize overfitting in the loss curves, and what is the first thing you do about it?